

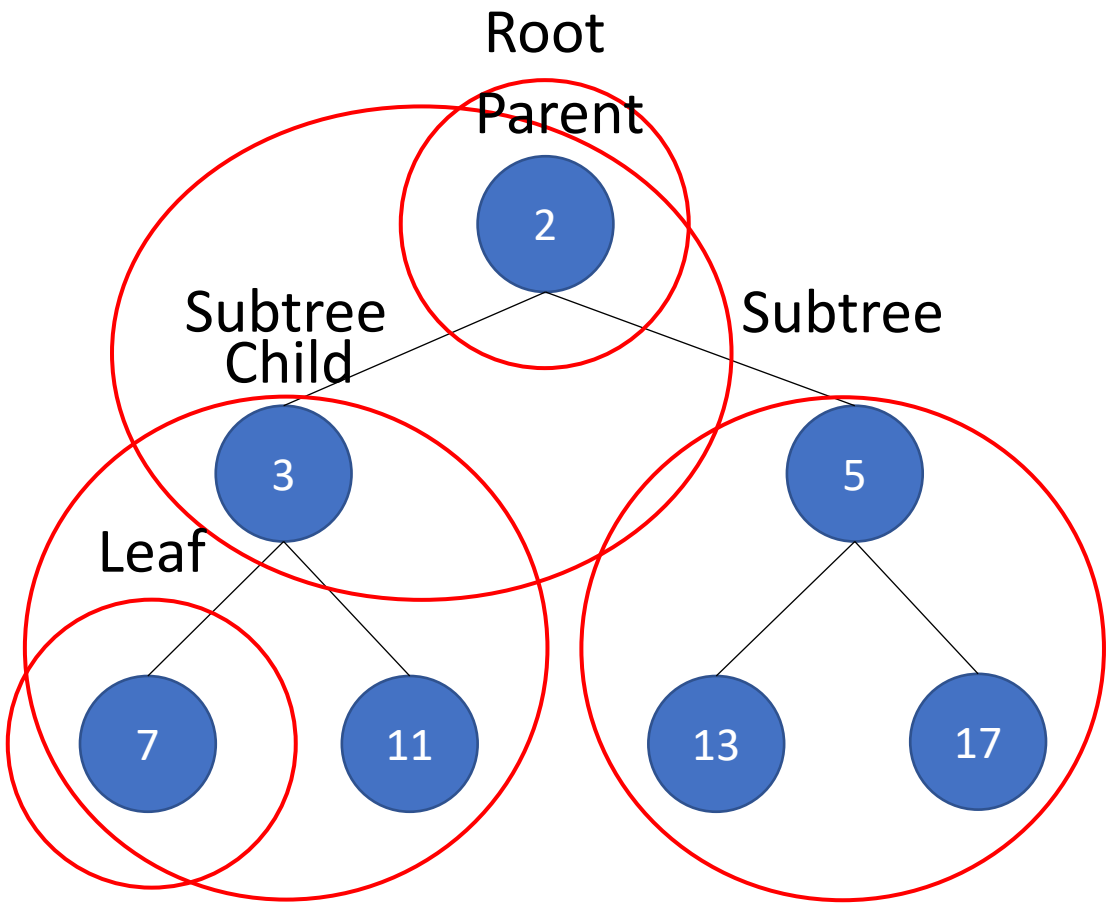
NANYANG TECHNOLOGICAL UNIVERSITY

Data Structures & Algorithms in Python

Binary Search Trees – Lab Answers

College of Computing and Data Science

Binary Trees Recap





Content Copyright Nanyang Technological University

Binary Trees Recap

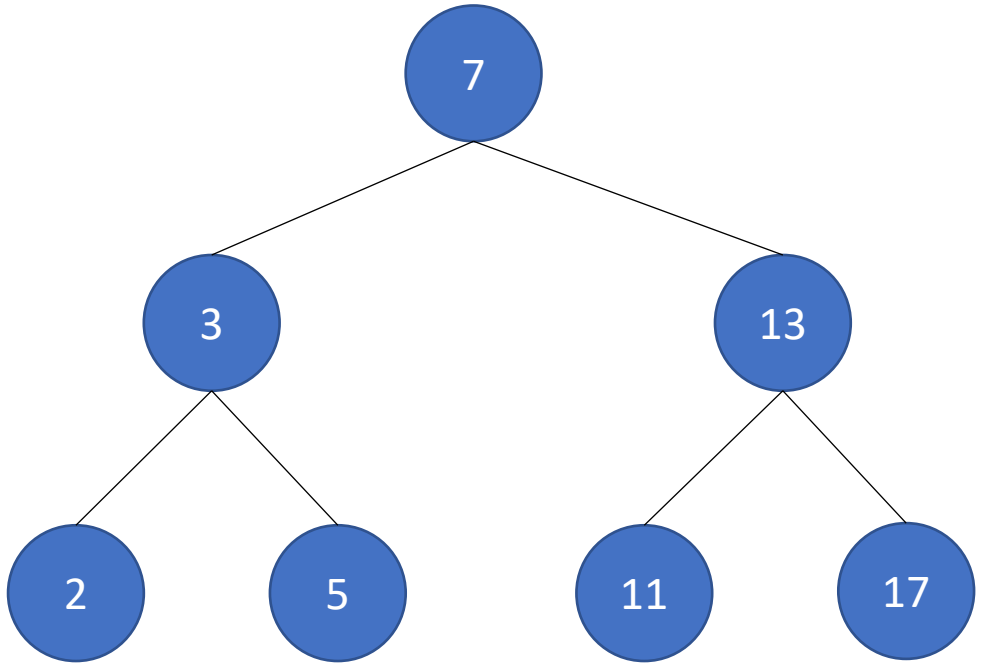
Tree traversal:

- Breadth-First Search
- Depth-First Search
  - Preorder
  - In-order
  - Postorder

Content Copyright Nanyang Technological University

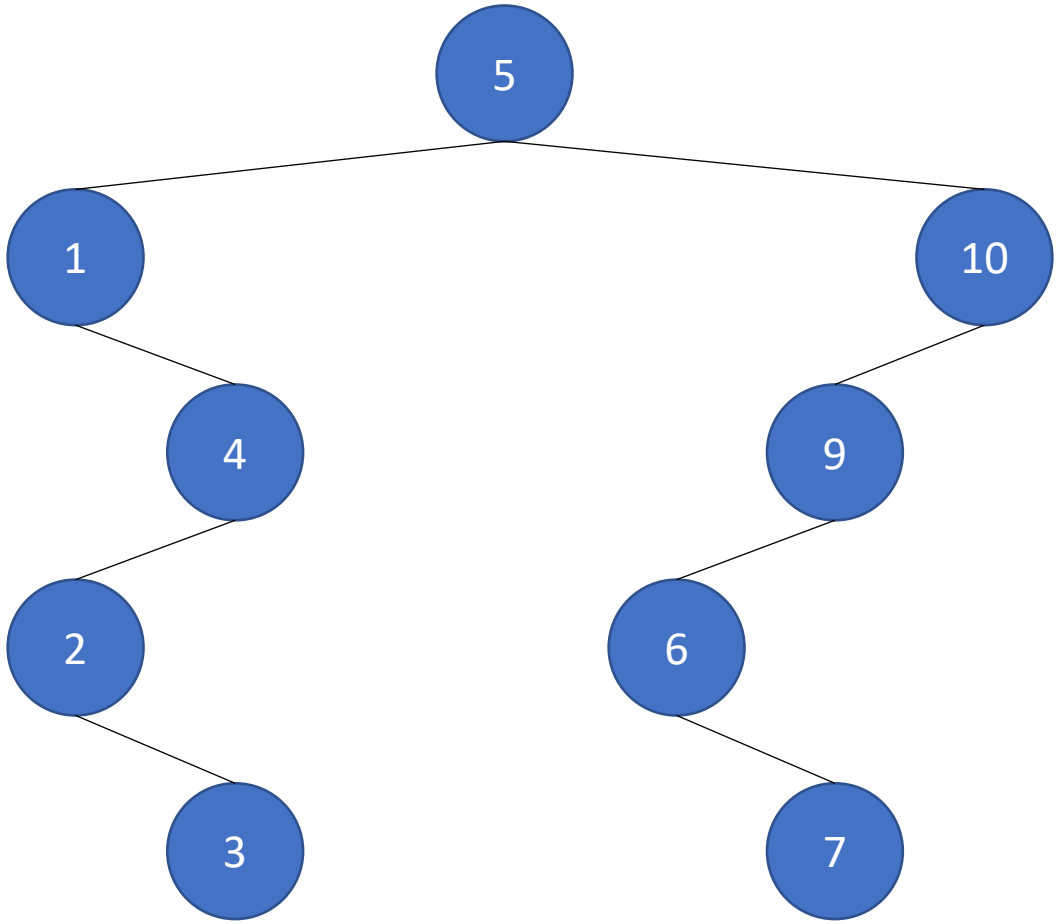
Binary Search Trees

- Node's value is greater than all values in its left subtree
- Node's value is lesser than all values in its right subtree
- Both subtrees of the node are also binary search trees



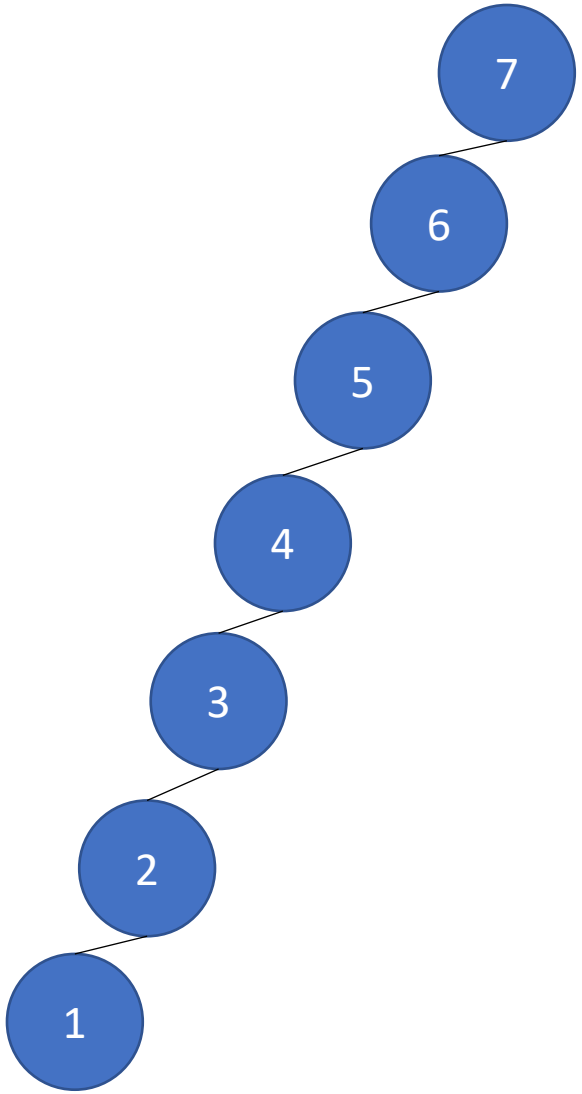
Content Copyright Nanyang Technological University

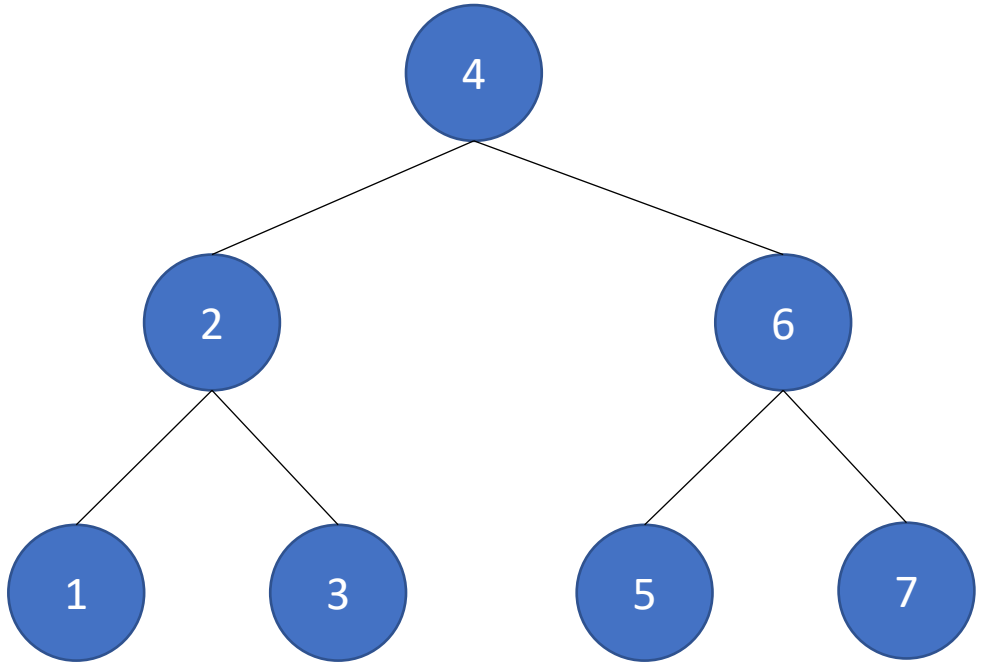
Binary Search Trees



Content Copyright Nanyang Technological University

Binary Search Trees





Content Copyright Nanyang Technological University

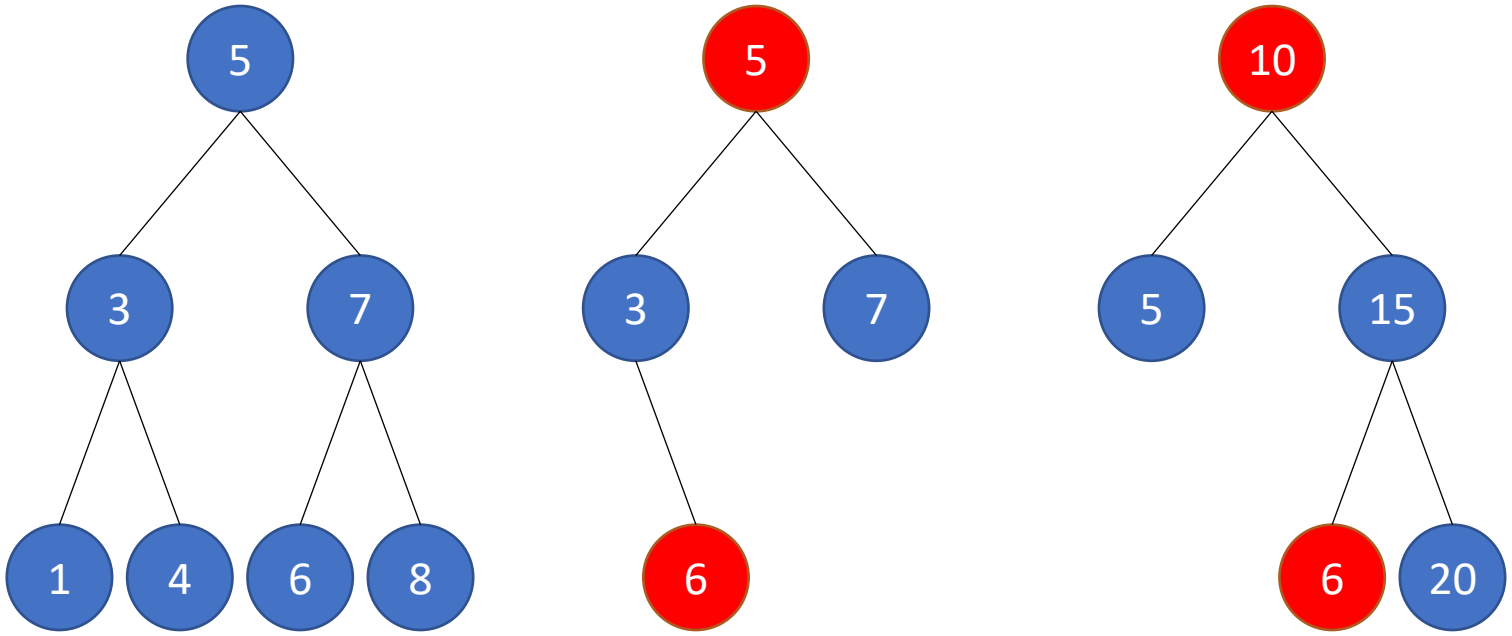
Q1 (No AI)

Given the root of a binary tree, determine whether it is a valid Binary Search Tree (BST). A tree is a valid BST if, for every node, all values in its left subtree are strictly less than the node's value and all values in its right subtree are strictly greater. This must hold for every node in the tree, not just for its immediate children.

Examples:
Input: [5, 3, 7, 1, 4, 6, 8]
Every node satisfies the BST property.
Output: True (Valid BST)

Input: [5, 3, 7, None, 6, None, None]
Node 6 is in the left subtree of 5,
but 6 > 5, violating the BST property.
Output: False (Not a valid BST)

Input: [10, 5, 15, None, None, 6, 20]
Node 6 is in the right subtree of 10,
but 6 < 10, violating the BST property.
Output: False (Not a valid BST)



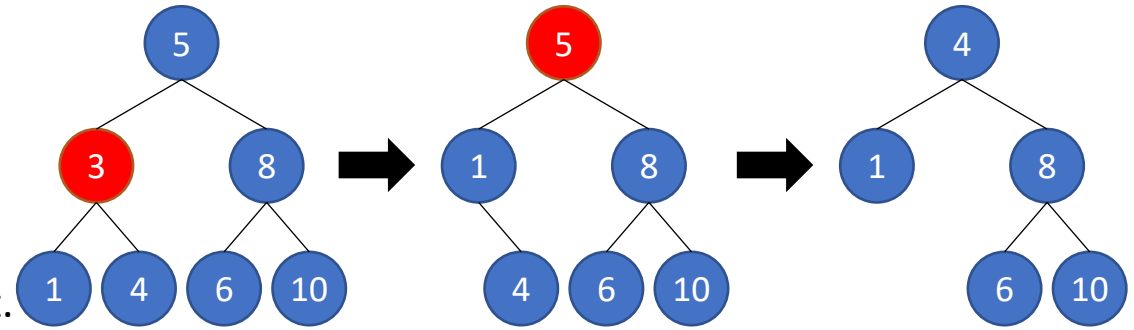
Content Copyright Nanyang Technological University

Q2 (Normal)

Given the root of a Binary Search Tree (BST) and a *value*, remove the node holding that value while keeping the BST property. There are three cases:

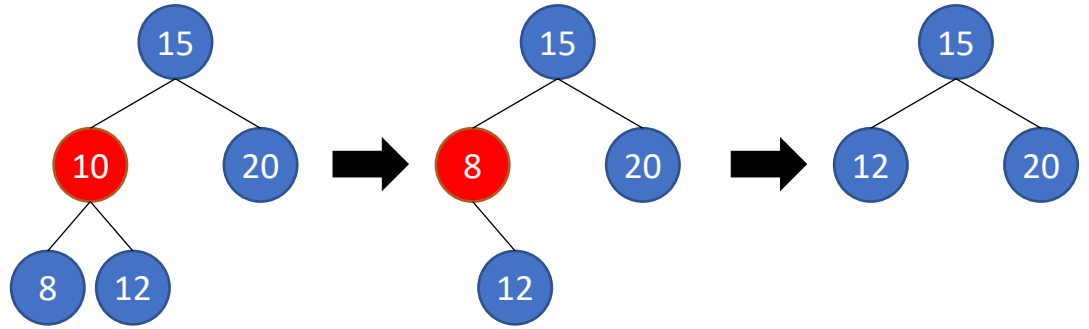
- **No children**: simply remove the node
- **One child**: replace the node with its child
- **Two children**: replace the node's value with its in-order predecessor (the largest value in its left subtree), then remove the predecessor from the left subtree.

Examples:
Input: [5, 3, 8, 1, 4, 6, 10]
Remove 3 Node 3 has two children (1 and 4).
The in-order predecessor is 1. Replace 3 with 1.
Output: [5, 1, 8, null, 4, 6, 10]



Input: [10, 5, 20]
Remove 5 Node 5 has no children. Simply remove it.
Output: [10, null, 20]

Input: [15, 10, 20, 8, 12]
Remove 10 Node 10 has two children (8 and 12).
The in-order predecessor is 8. Replace 10 with 8.
Output: [15, 8, 20, null, 12]



Input: [2]
Remove 2 Node 2 is the root with no children.
Tree becomes empty.
Output: []

Content Copyright Nanyang Technological University

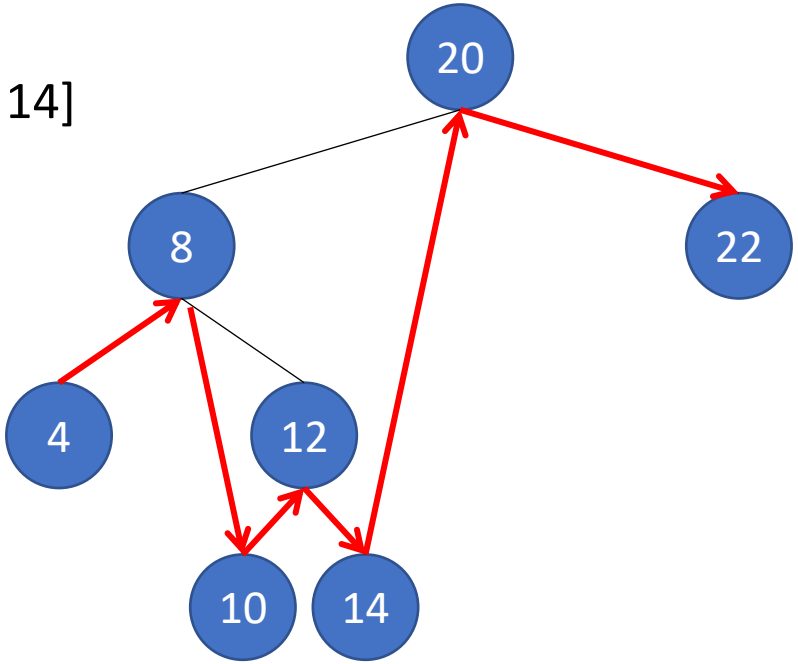
Q3 (Driver)

Given the root of a Binary Search Tree (BST) and an integer k, return the k-th smallest value in the tree (1-indexed). Because it is a BST, an in-order traversal visits the nodes in ascending order, so the k-th node visited is the answer.

Your solution should use O(h) extra space, where h is the height of the tree.

Examples:
Input: [20, 8, 22, 4, 12, None, None, None, None, 10, 14]
In-order traversal: 4, 8, 10, 12, 14, 20, 22
k=1 → 4 k=3 → 10 k=5 → 14 k=7 → 22

Input: [5, 3, 6, 2, 4, None, None, 1]
In-order traversal: 1, 2, 3, 4, 5, 6
k=1 → 1 k=3 → 3 k=6 → 6

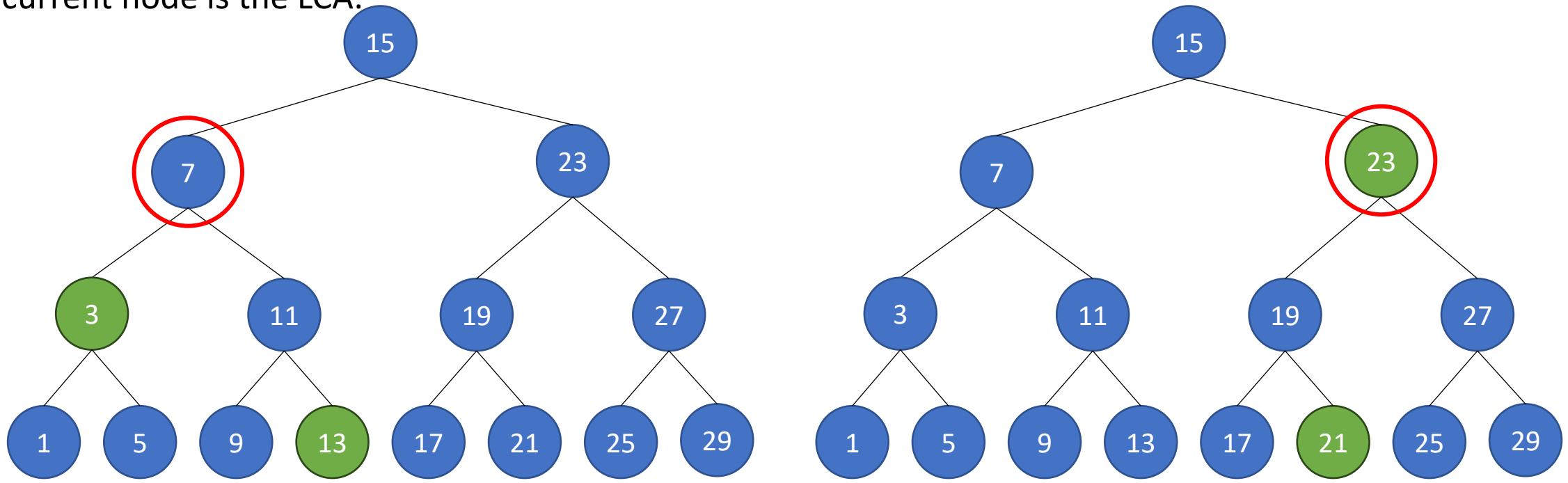


Content Copyright Nanyang Technological University

Q4 (Navigator)

Given the root of a Binary Search Tree (BST) and two values p and q that are both in the tree, find their lowest common ancestor (LCA): the lowest node in the tree that has both p and q as descendants, where a node is allowed to be a descendant of itself.

Use the BST property to decide which way to walk: if both values are smaller than the current node, the LCA lies in its left subtree; if both are larger, it lies in its right subtree; otherwise the current node is the LCA.



Content Copyright Nanyang Technological University

Lab Access Code

SR3-L4-YG7T

Content Copyright Nanyang Technological University

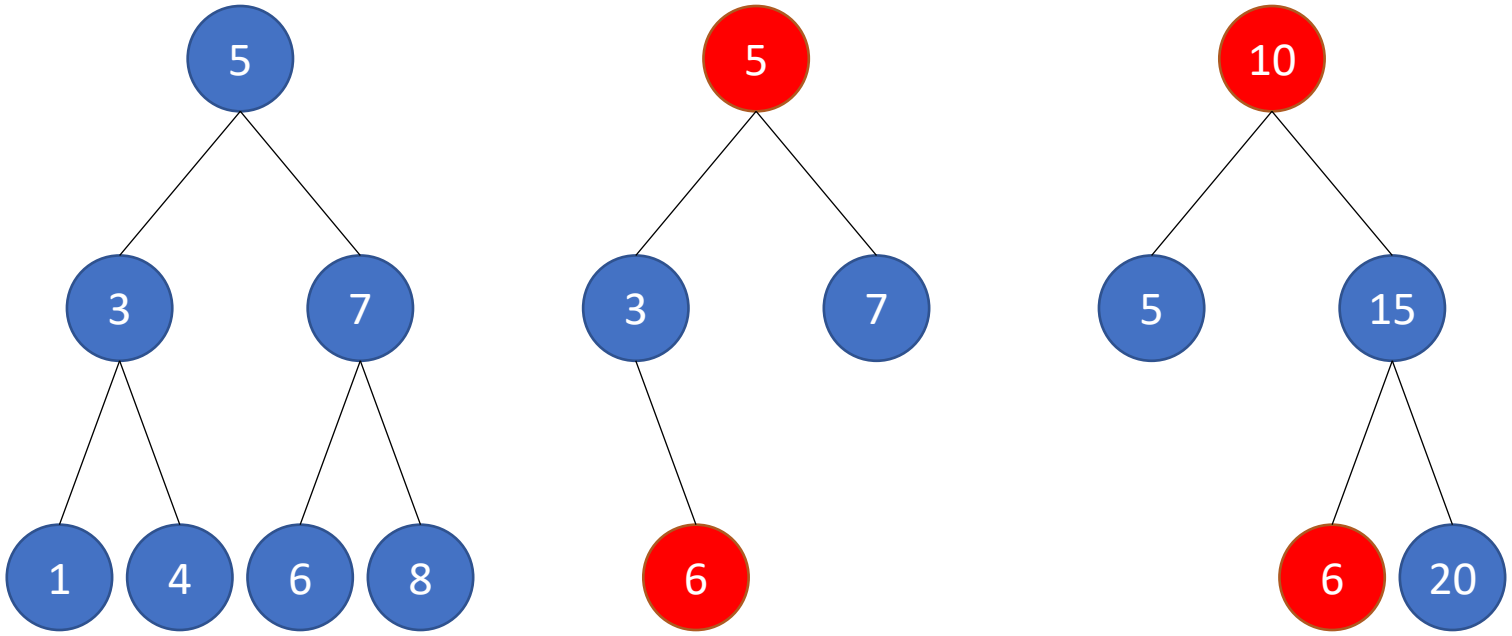
Q1 (No AI) – Answer

Given the root of a binary tree, determine whether it is a valid Binary Search Tree (BST). A tree is a valid BST if, for every node, all values in its left subtree are strictly less than the node's value and all values in its right subtree are strictly greater. This must hold for every node in the tree, not just for its immediate children.

Examples:
Input: [5, 3, 7, 1, 4, 6, 8]
Every node satisfies the BST property.
Output: True (Valid BST)

Input: [5, 3, 7, None, 6, None, None]
Node 6 is in the left subtree of 5,
but 6 > 5, violating the BST property.
Output: False (Not a valid BST)

Input: [10, 5, 15, None, None, 6, 20]
Node 6 is in the right subtree of 10,
but 6 < 10, violating the BST property.
Output: False (Not a valid BST)



Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def isBST(node, min_val=float('-inf'), max_val=float('inf')):
    """ Checks if a tree is a valid BST using min/max
    constraints. """
    if node is None:
        return True

    if not (min_val < node.item < max_val):
        return False

    return isBST(node.left, min_val, node.item)
    and isBST(node.right, node.item, max_val)
```

```
def isBST(self, root):
    def helper(node, min_val, max_val):
        if node is None:
            return True
        if node.data <= min_val or node.data >= max_val:
            return False
        return helper(node.left, min_val, node.data)
        and helper(node.right, node.data, max_val)
    return helper(root, float('-inf'), float('inf'))
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

If node is None return True

```
def isBST(self, root):
    def helper(node, min_val, max_val):
        if node is None:
            return True
        if node.data <= min_val or node.data >= max_val:
            return False
        return helper(node.left, min_val, node.data)
        and helper(node.right, node.data, max_val)
    return helper(root, float('-inf'), float('inf'))
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

If node is None return True

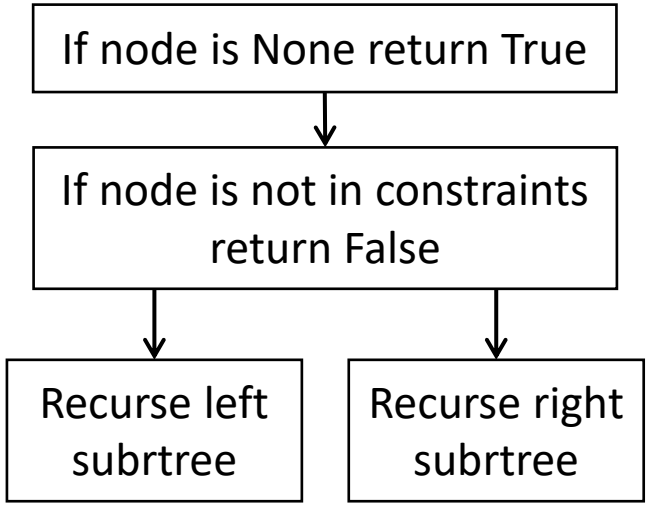
↓

If node is not in constraints return False

```
def isBST(self, root):
    def helper(node, min_val, max_val):
        if node is None:
            return True
        if node.data <= min_val or node.data >= max_val:
            return False
        return helper(node.left, min_val, node.data)
        and helper(node.right, node.data, max_val)
    return helper(root, float('-inf'), float('inf'))
```

Content Copyright Nanyang Technological University

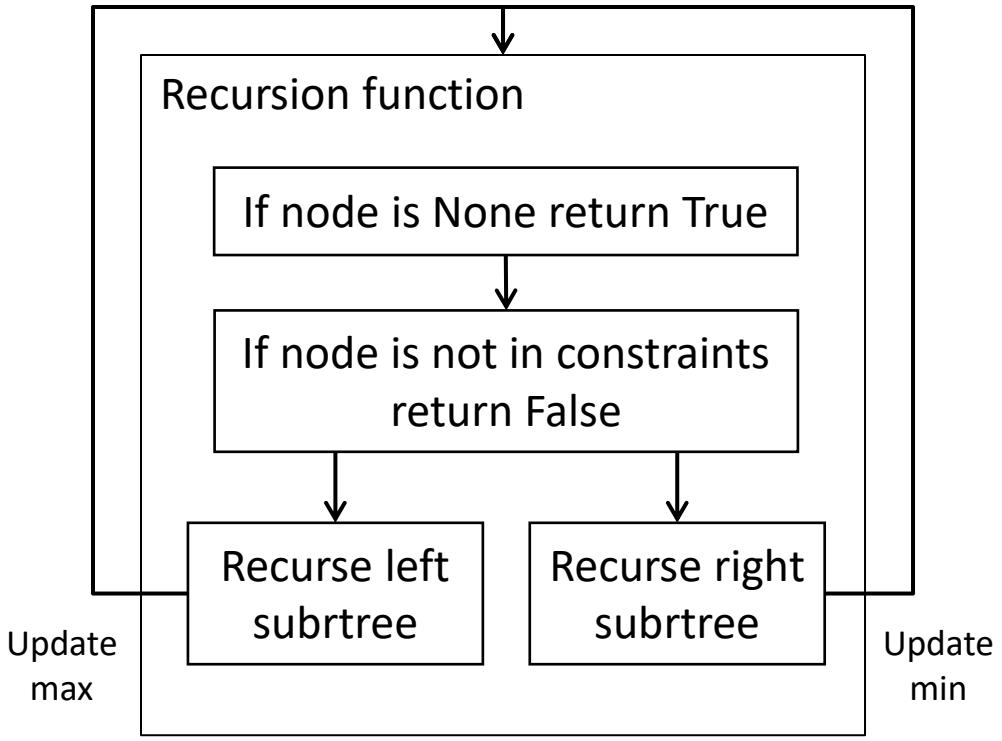
Q1 (No AI) – Answer




```
def isBST(self, root):
    def helper(node, min_val, max_val):
        if node is None:
            return True
        if node.data <= min_val or node.data >= max_val:
            return False
        return helper(node.left, min_val, node.data)
        and helper(node.right, node.data, max_val)
    return helper(root, float('-inf'), float('inf'))
```

Content Copyright Nanyang Technological University

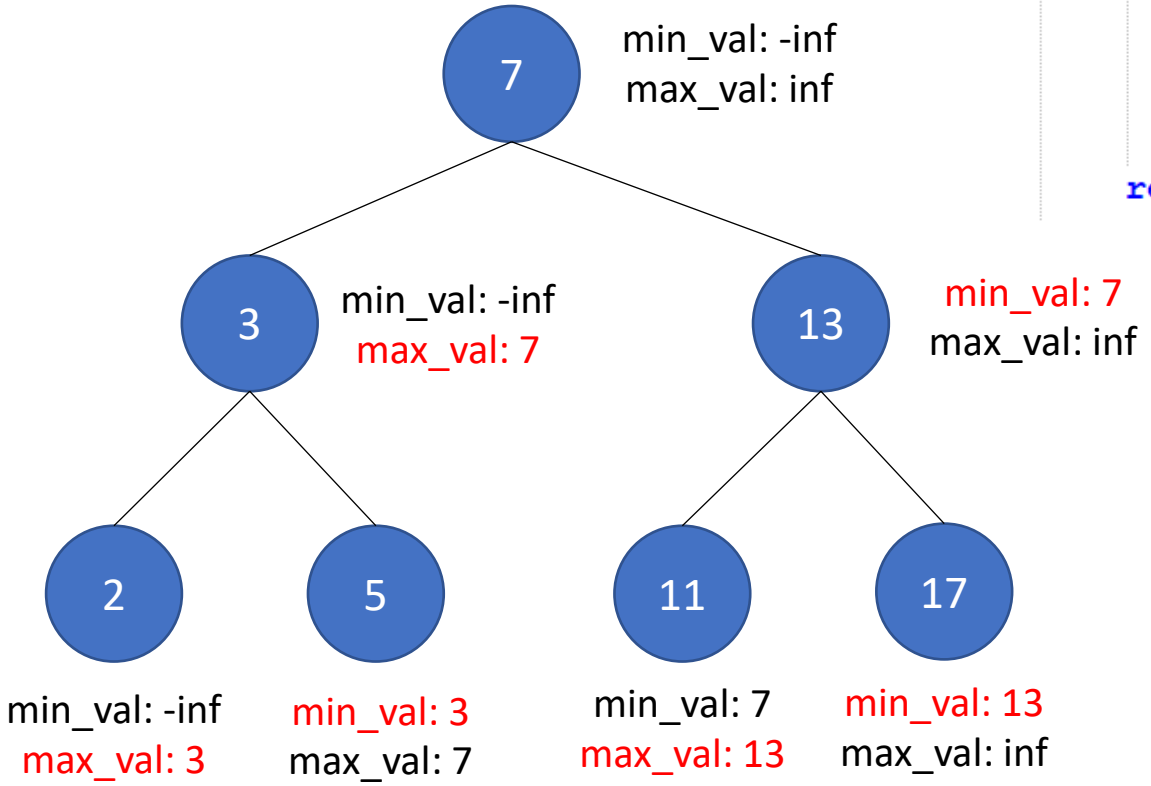
Q1 (No AI) – Answer




```
def isBST(self, root):
    def helper(node, min_val, max_val):
        if node is None:
            return True
        if node.data <= min_val or node.data >= max_val:
            return False
        return helper(node.left, min_val, node.data)
        and helper(node.right, node.data, max_val)
    return helper(root, float('-inf'), float('inf'))
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer




```python
def isBST(self, root):
    def helper(node, min_val, max_val):
        if node is None:
            return True
        if node.data <= min_val or node.data >= max_val:
            return False
        return helper(node.left, min_val, node.data)
        and helper(node.right, node.data, max_val)
    return helper(root, float('-inf'), float('inf'))
```

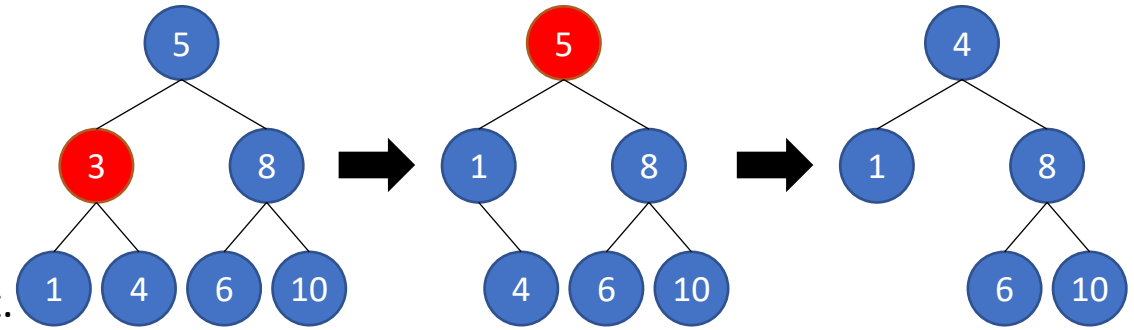
Content Copyright Nanyang Technological University

Q2 (Normal)

Given the root of a Binary Search Tree (BST) and a *value*, remove the node holding that value while keeping the BST property. There are three cases:

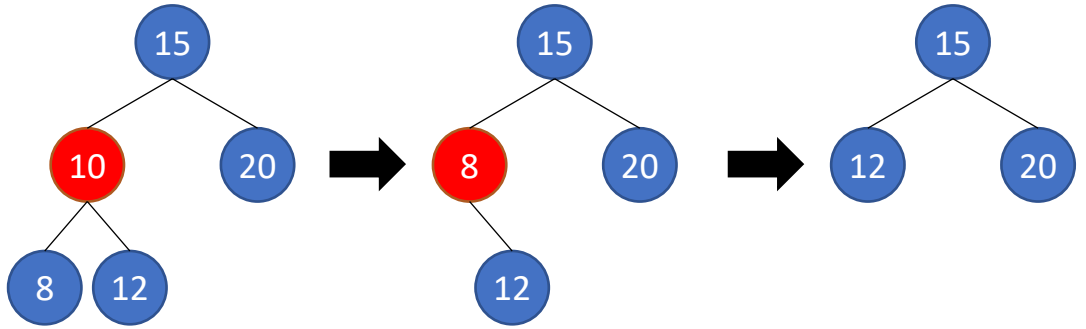
- **No children**: simply remove the node
- **One child**: replace the node with its child
- **Two children**: replace the node's value with its in-order predecessor (the largest value in its left subtree), then remove the predecessor from the left subtree.

Examples:
Input: [5, 3, 8, 1, 4, 6, 10]
Remove 3 Node 3 has two children (1 and 4).
The in-order predecessor is 1. Replace 3 with 1.
Output: [5, 1, 8, null, 4, 6, 10]



Input: [10, 5, 20]
Remove 5 Node 5 has no children. Simply remove it.
Output: [10, null, 20]

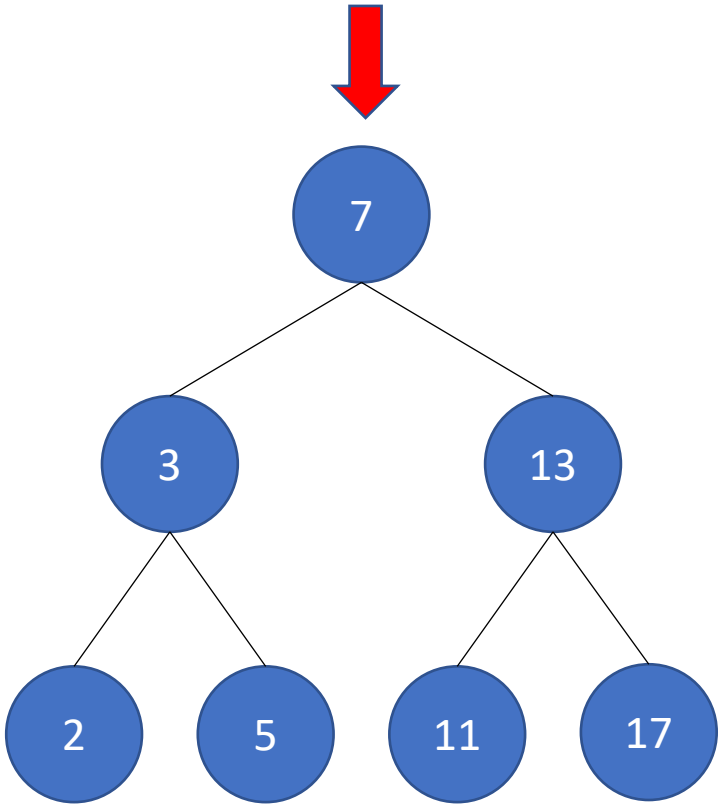
Input: [15, 10, 20, 8, 12]
Remove 10 Node 10 has two children (8 and 12).
The in-order predecessor is 8. Replace 10 with 8.
Output: [15, 8, 20, null, 12]



Input: [2]
Remove 2 Node 2 is the root with no children.
Tree becomes empty.
Output: []

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

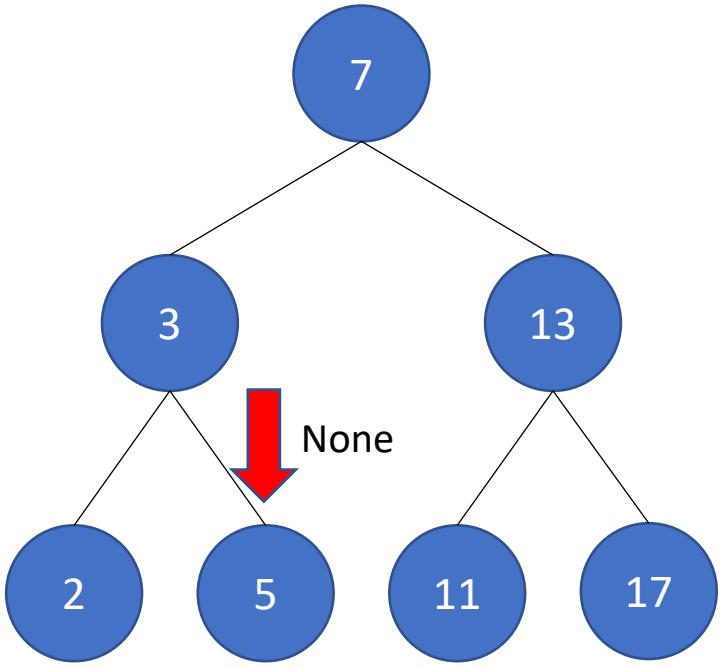


Value = 5

```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

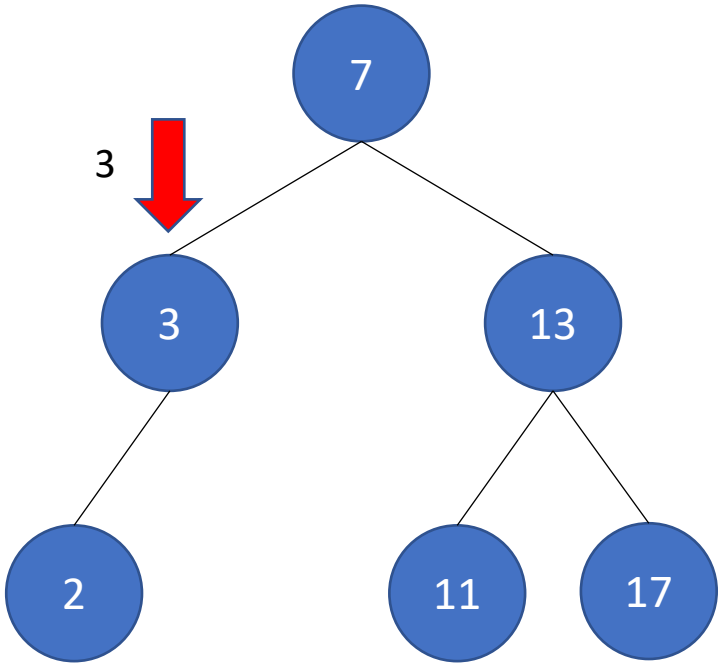


Value = 5

```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

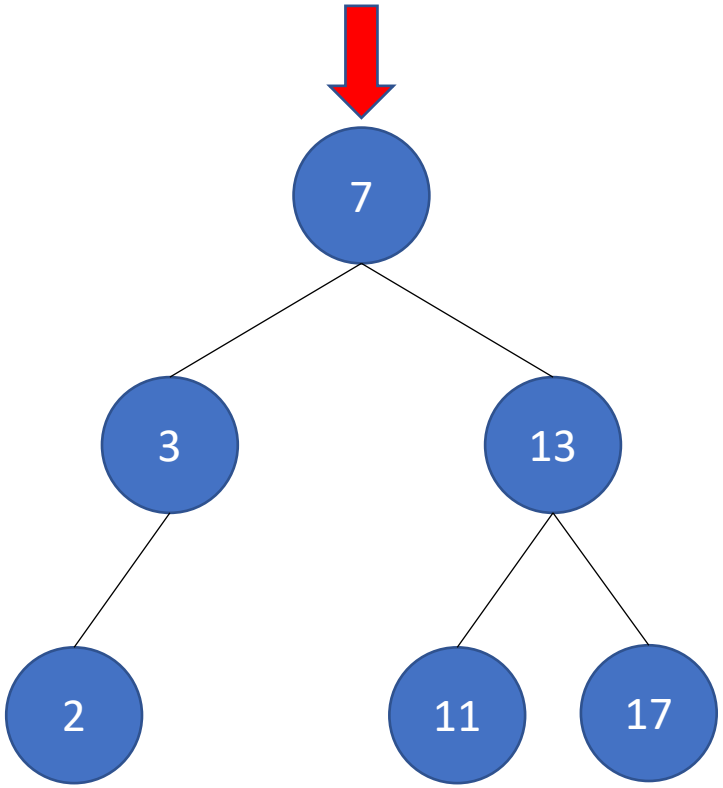


Value = 5

```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer



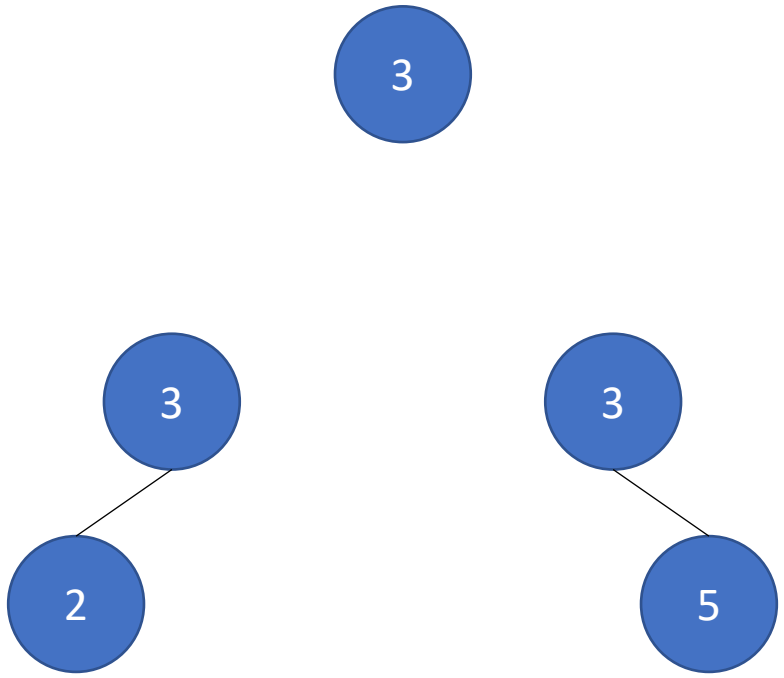
Value = 5

```
root = Solution().removeBSTNode(root, i)


def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

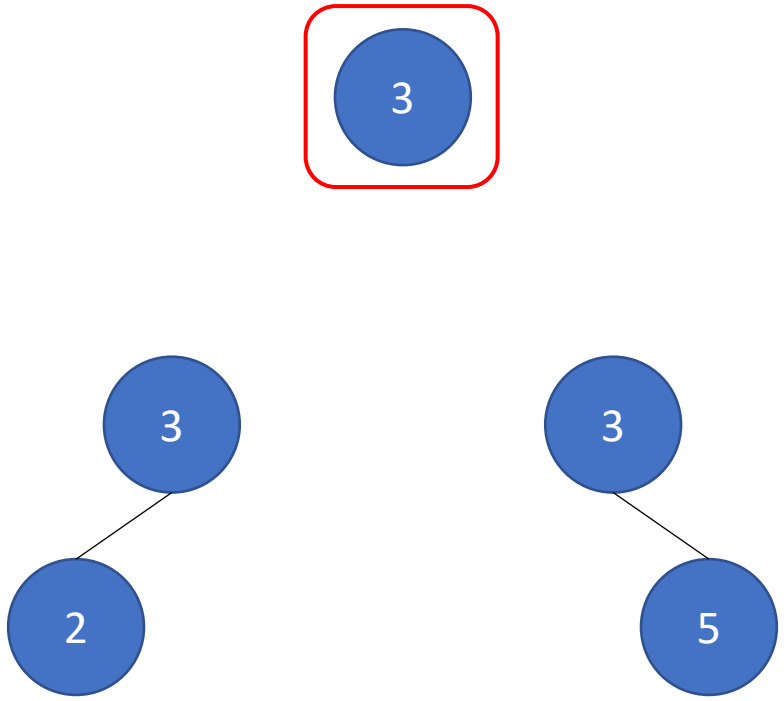


Value = 3

```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

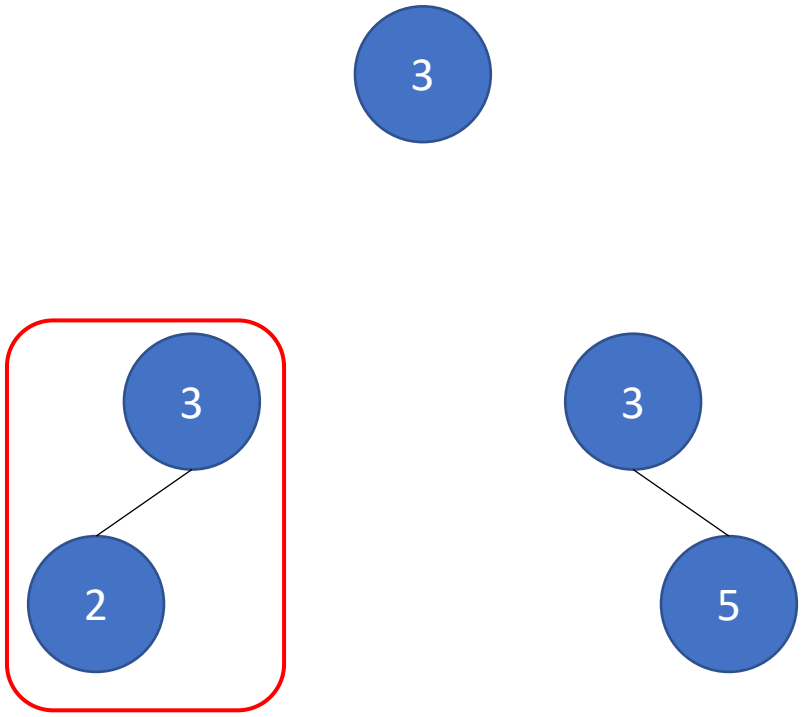


Value = 3

```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

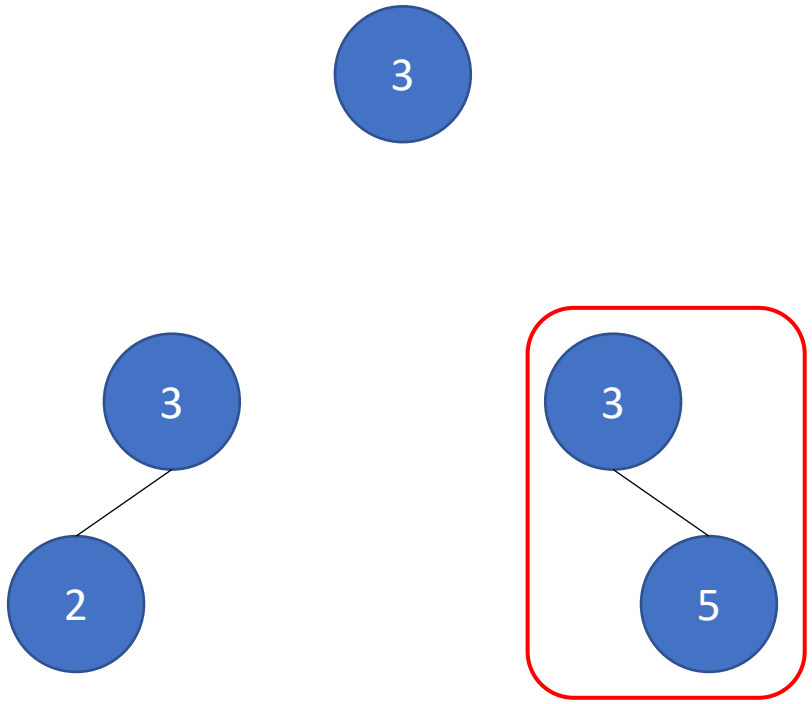


Value = 3

```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer



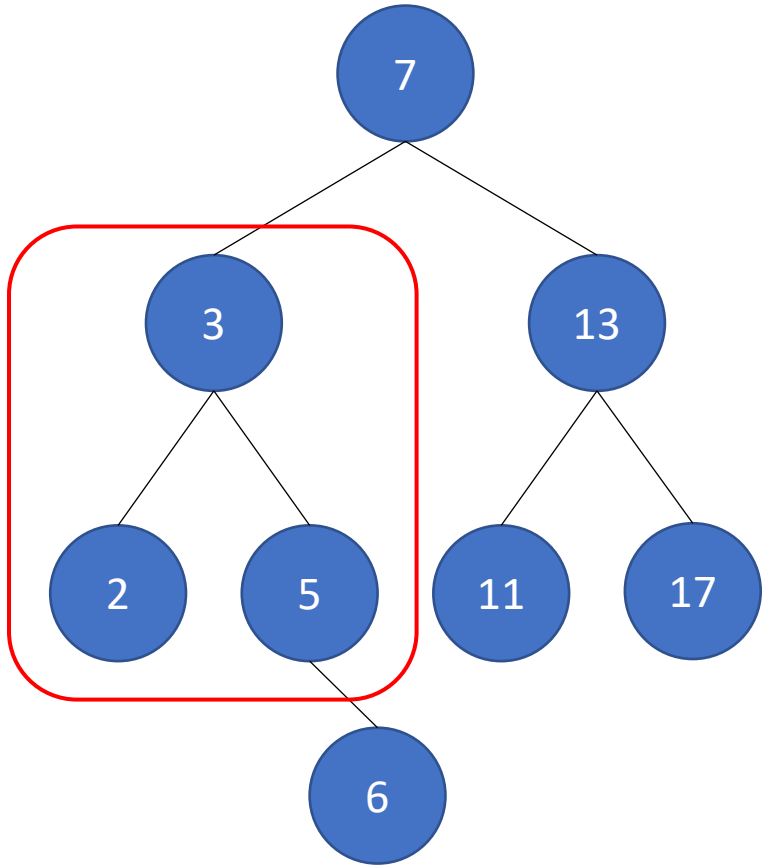

```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Value = 3

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

Value = 7



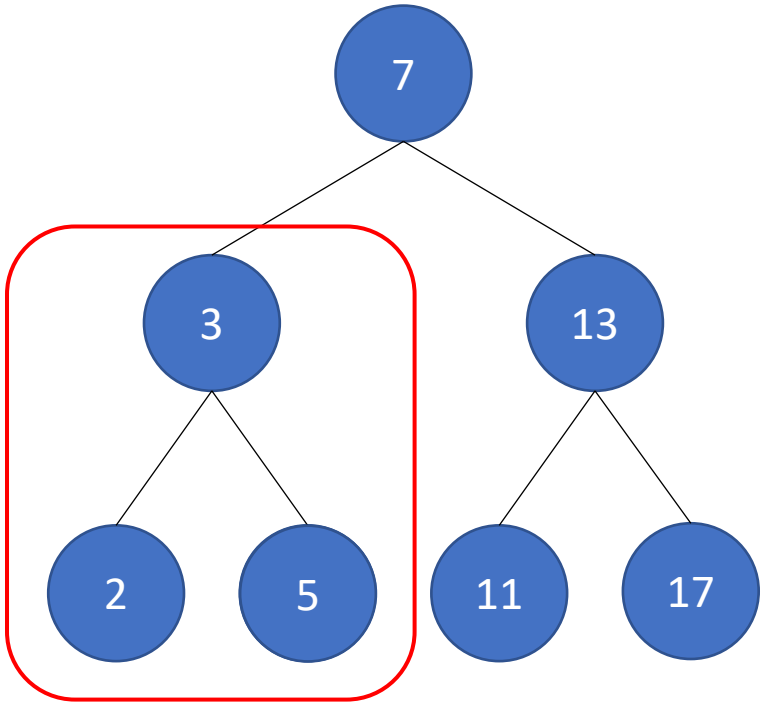

```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Rightmost node of the left subtree

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

Value = 7




```
def removeBSTNode(self, root, value):
    if root is None:
        return None
    if value < root.data:
        root.left = self.removeBSTNode(root.left, value)
    elif value > root.data:
        root.right = self.removeBSTNode(root.right, value)
    else:  # Found the node to remove
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            predecessor = root.left
            while predecessor.right is not None:
                predecessor = predecessor.right
            root.data = predecessor.data
            root.left = self.removeBSTNode(root.left, predecessor.data)
    return root
```

Content Copyright Nanyang Technological University

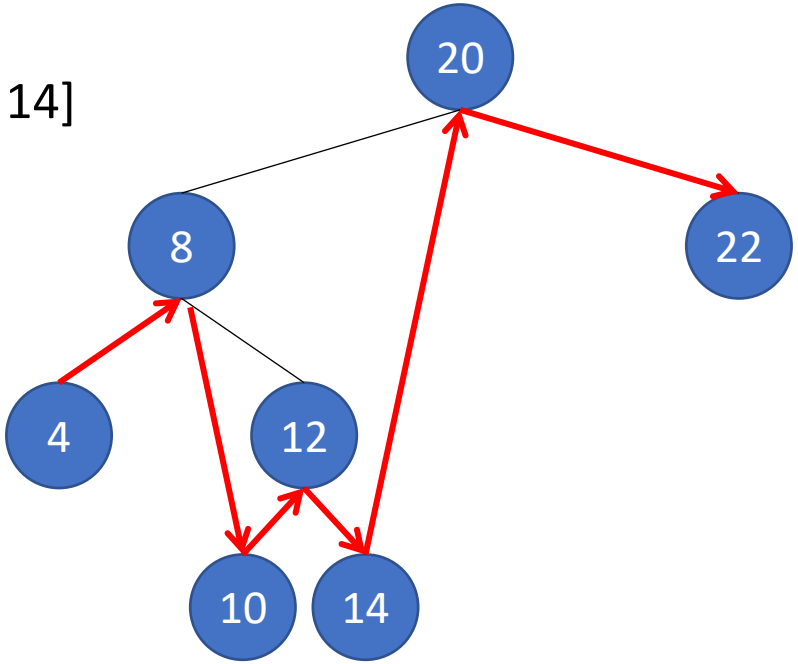
Q3 (Driver) – Answer

Given the root of a Binary Search Tree (BST) and an integer k, return the k-th smallest value in the tree (1-indexed). Because it is a BST, an in-order traversal visits the nodes in ascending order, so the k-th node visited is the answer.

Your solution should use O(h) extra space, where h is the height of the tree.

Examples:
Input: [20, 8, 22, 4, 12, None, None, None, None, 10, 14]
In-order traversal: 4, 8, 10, 12, 14, 20, 22
k=1 → 4 k=3 → 10 k=5 → 14 k=7 → 22

Input: [5, 3, 6, 2, 4, None, None, 1]
In-order traversal: 1, 2, 3, 4, 5, 6
k=1 → 1 k=3 → 3 k=6 → 6



Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def kthSmallest(self, root, k):
    count = 0
    stack = []
    current = root
    while current or stack:
        while current:
            stack.append(current)
            current = current.left
        current = stack.pop()
        count += 1
        if count == k:
            return current.data
        current = current.right
    return -1
```

```
stack = []
curr = root
n = 0  # Counter for elements visited

while stack or curr:
    # 1. Reach the left-most node of the current subtree
    while curr:
        stack.append(curr)
        curr = curr.left

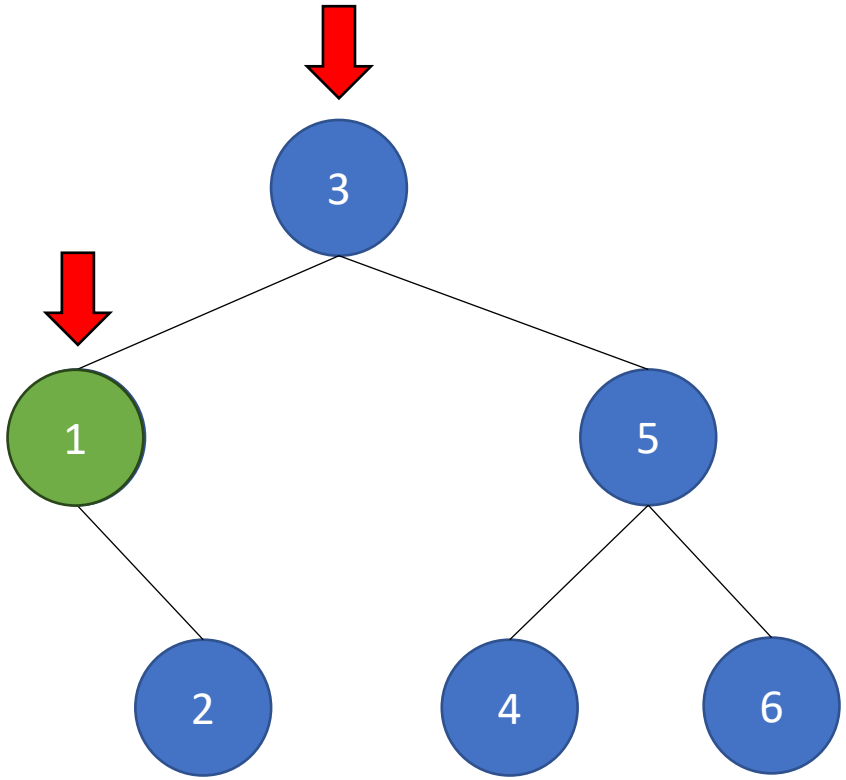
    # 2. Process the node
    curr = stack.pop()
    n += 1
    if n == k:
        return curr.val

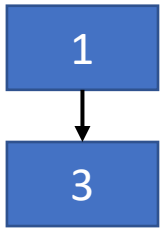
    # 3. Move to the right subtree
    curr = curr.right

return -1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer





n = 0

```
stack = []
curr = root
n = 0  # Counter for elements visited

while stack or curr:
    # 1. Reach the left-most node of the current subtree
    while curr:
        stack.append(curr)
        curr = curr.left

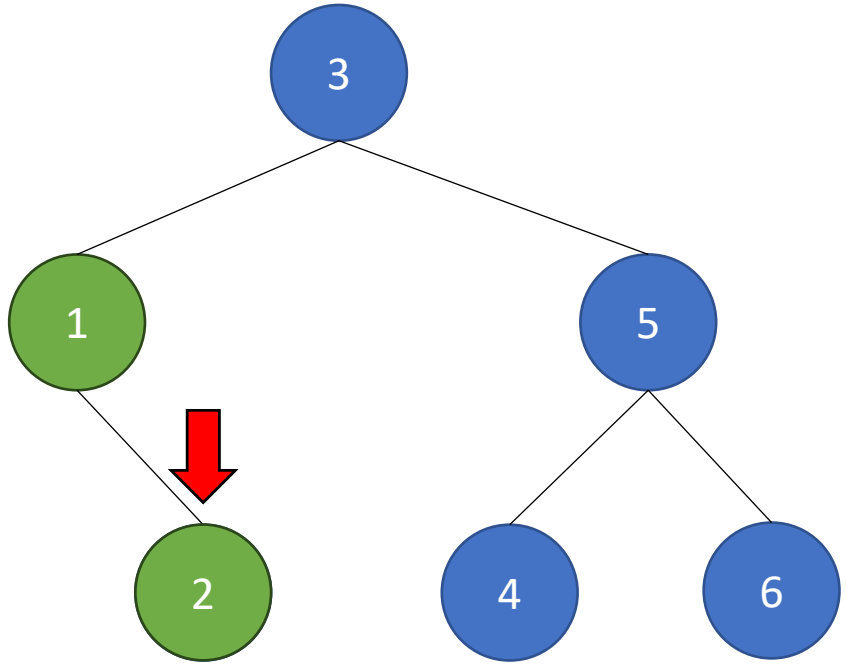
    # 2. Process the node
    curr = stack.pop()
    n += 1
    if n == k:
        return curr.val

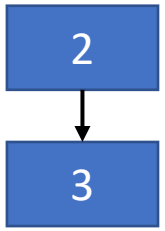
    # 3. Move to the right subtree
    curr = curr.right

return -1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer





n = 2

```
stack = []
curr = root
n = 0  # Counter for elements visited

while stack or curr:
    # 1. Reach the left-most node of the current subtree
    while curr:
        stack.append(curr)
        curr = curr.left

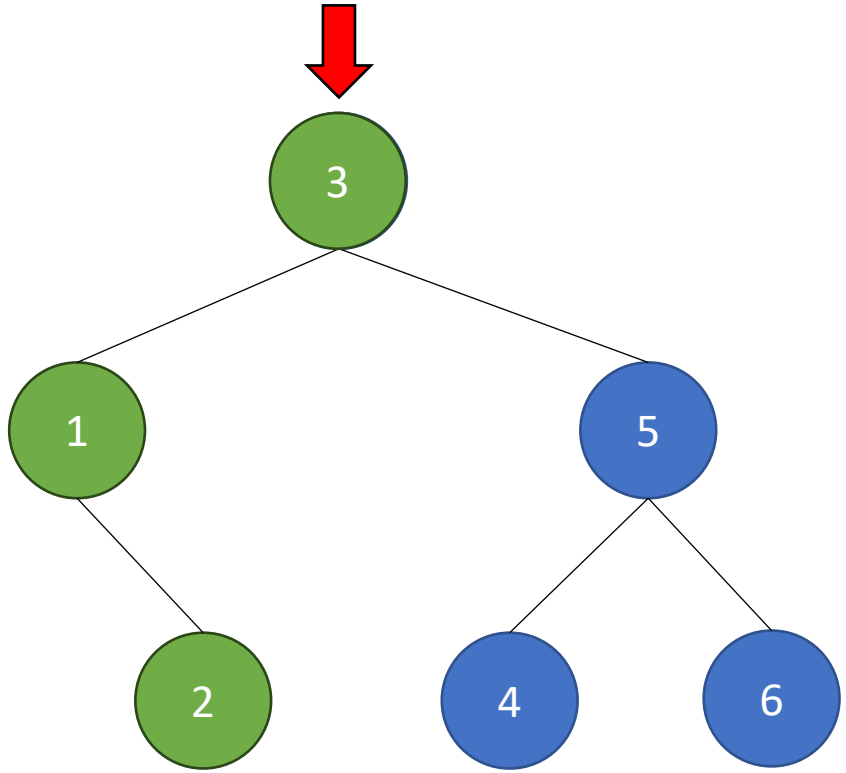
    # 2. Process the node
    curr = stack.pop()
    n += 1
    if n == k:
        return curr.val

    # 3. Move to the right subtree
    curr = curr.right

return -1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer



n = 3

3

```
stack = []
curr = root
n = 0  # Counter for elements visited

while stack or curr:
    # 1. Reach the left-most node of the current subtree
    while curr:
        stack.append(curr)
        curr = curr.left

    # 2. Process the node
    curr = stack.pop()
    n += 1
    if n == k:
        return curr.val

    # 3. Move to the right subtree
    curr = curr.right

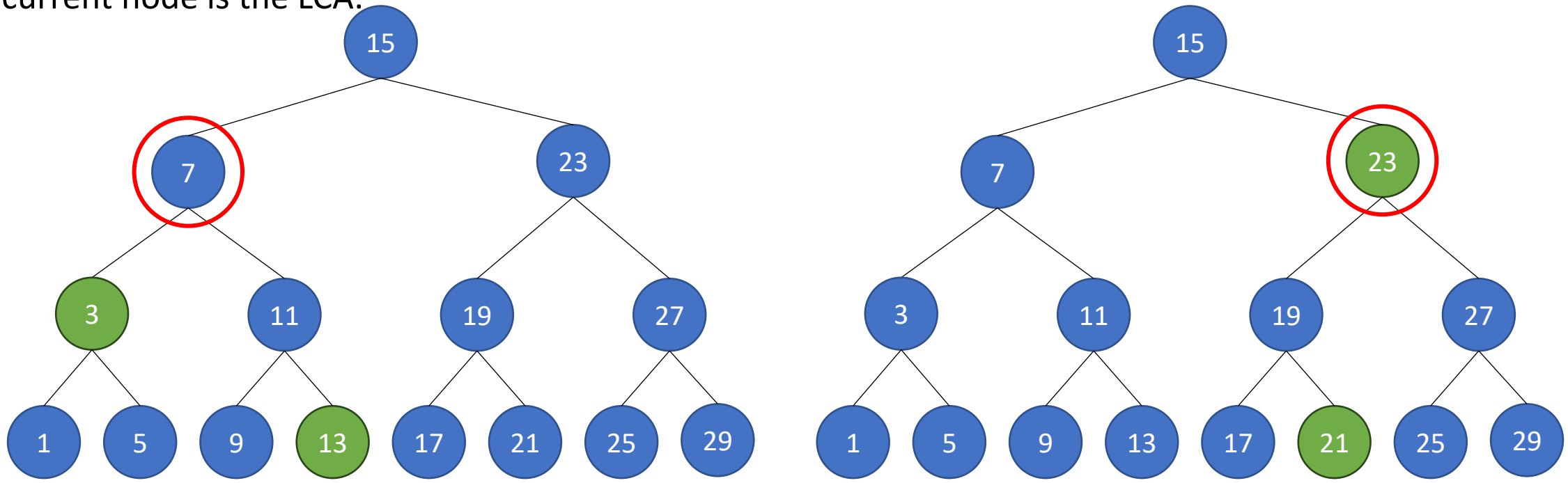
return -1
```

Content Copyright Nanyang Technological University

Q4 (Navigator)

Given the root of a Binary Search Tree (BST) and two values p and q that are both in the tree, find their lowest common ancestor (LCA): the lowest node in the tree that has both p and q as descendants, where a node is allowed to be a descendant of itself.

Use the BST property to decide which way to walk: if both values are smaller than the current node, the LCA lies in its left subtree; if both are larger, it lies in its right subtree; otherwise the current node is the LCA.



Content Copyright Nanyang Technological University

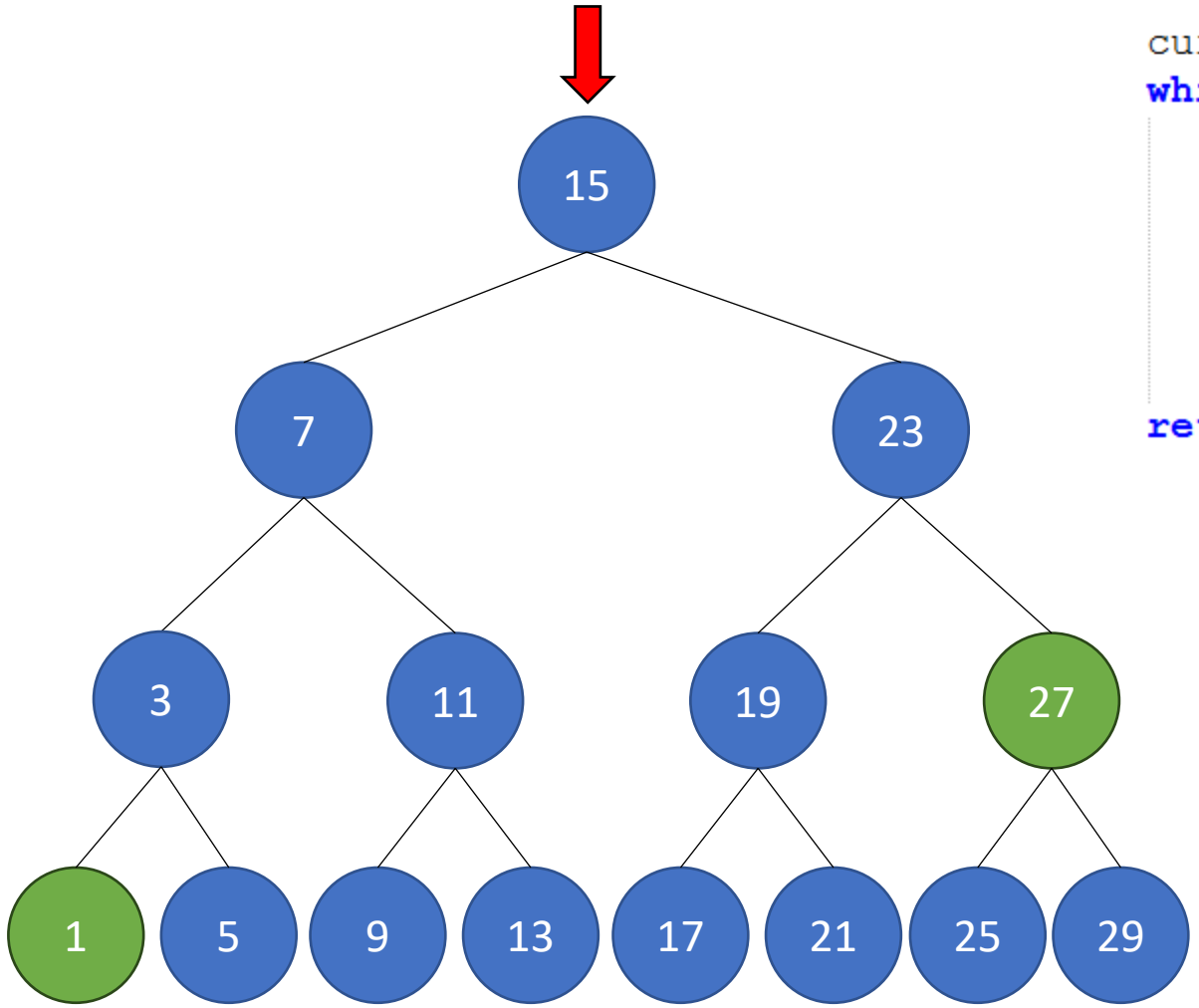
Q4 (Navigator) – Answer

```
def findLCA(self, root, p, q):
    if root is None:
        return -1
    if p < root.data and q < root.data:
        return self.findLCA(root.left, p, q)
    elif p > root.data and q > root.data:
        return self.findLCA(root.right, p, q)
    else:
        return root.data
```

```
current = root
while current:
    if p < current.value and q < current.value:
        current = current.left
    elif p > current.value and q > current.value:
        current = current.right
    else:
        return current.value
return -1
```

Content Copyright Nanyang Technological University

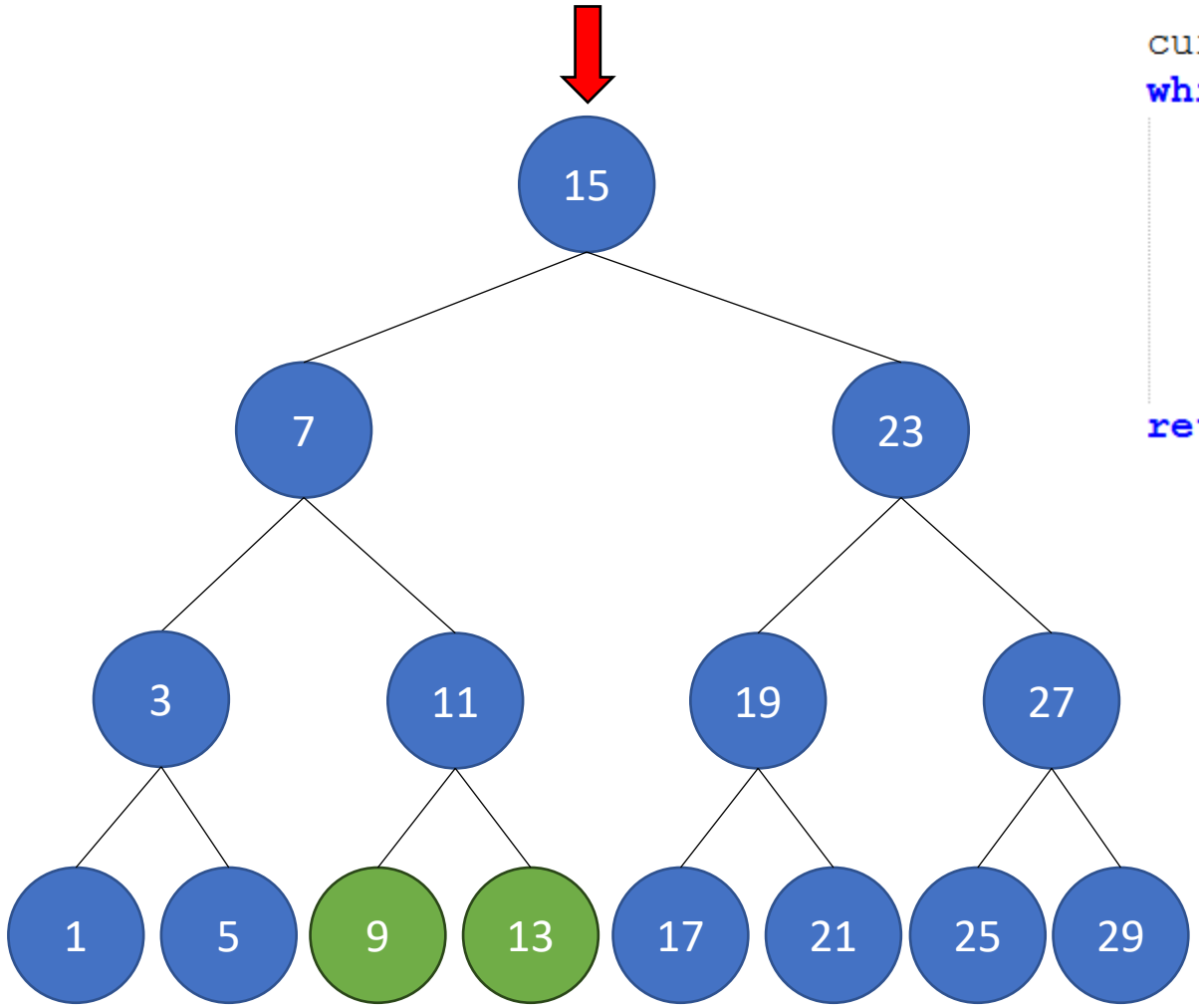
Q4 (Navigator) – Answer




```
current = root
while current:
    if p < current.value and q < current.value:
        current = current.left
    elif p > current.value and q > current.value:
        current = current.right
    else:
        return current.value
return -1
```

Content Copyright Nanyang Technological University

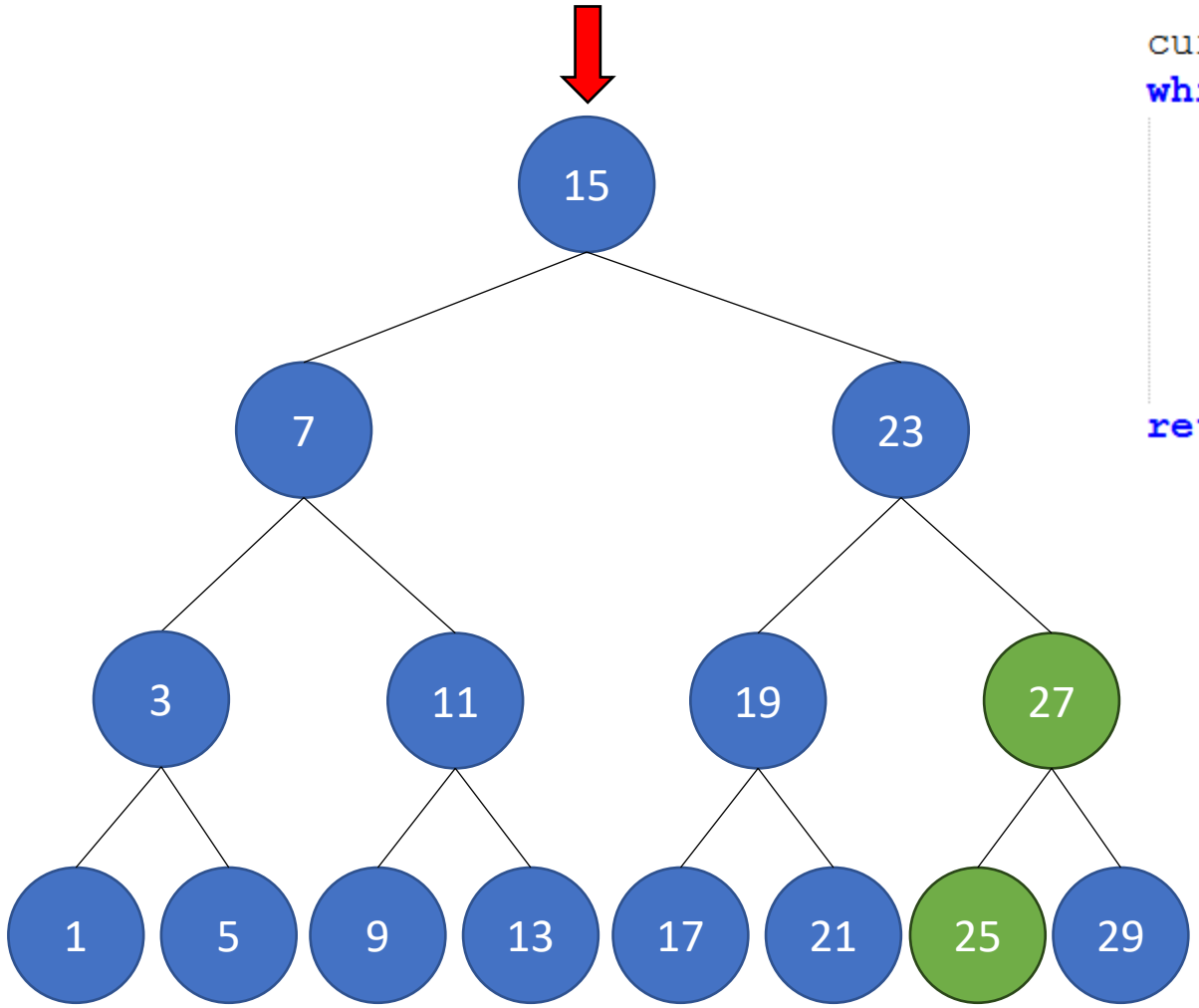
Q4 (Navigator) – Answer




```
current = root
while current:
    if p < current.value and q < current.value:
        current = current.left
    elif p > current.value and q > current.value:
        current = current.right
    else:
        return current.value
return -1
```

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer




```
current = root
while current:
    if p < current.value and q < current.value:
        current = current.left
    elif p > current.value and q > current.value:
        current = current.right
    else:
        return current.value
return -1
```

Content Copyright Nanyang Technological University